AGENTTX: Causal Effect Transactions for Long-Running Coding Agents

Anonymous submission

Abstract

Long-running coding agents execute opaque tools across repositories, compilers, and test suites, creating filesystem effects that remain provisional until validation decides which results may safely reach the host. Existing choices either expose mistakes, fragment cross-call state, or discard later work that is temporally subsequent but causally unrelated to a failure. The key insight is to organize an Agent Effect Transaction around causal dependencies among versioned effects, making causality the recovery unit inside one trajectory. AGENTTX realizes this insight through effect capture, a causal ledger, shared speculation, selective reconstruction, and durable publication. Persistent workers and incremental snapshots separately reduce long-trajectory overhead. Across 144 controlled runs, AGENTTX retained all independent work and removed all invalid descendants; temporal recovery retained only 41% at 64 calls. At the largest replay input, it avoided 1,336–2,891 tokens; its 148 ms/step path beat per-call isolation by 43% but remained 3× slower than bare.

1 Introduction

Coding agents increasingly behave as long-running systems programs rather than single-turn code generators. A task may inspect a repository, edit several modules, invoke compilers and tests, revise a failed approach, and publish only the verified result. The user sees one adaptive trajectory, while the operating system sees unrelated processes whose filesystem mutations become visible immediately. Because failures may be diagnosed many calls later, intermediate effects must remain provisional without erasing correct progress.

Existing mechanisms organize speculative work around a command, session, branch, or temporal checkpoint. Bare execution is fast, but every provisional mutation reaches the host before approval. Among isolated options, a *per-call sandbox* offers the finest native undo, yet fragments shared state and repeatedly rebuilds isolation. Sessions and branches preserve state, while temporal checkpoints retain a prefix; both discard later valid work when failures and independent work interleave [1, 5, 7].

Measurements expose both costs: per-call `try` takes 260.7 ms/step on a 64-call workload, while unsafe bare execution takes 49.7 ms/step and pollutes the host. On a controlled 64-call DAG, temporal rollback retained only 41% of independent work, while omitting dependencies removed only 4% of invalid descendants. *Dependency discovery* must recover producer–consumer edges from opaque subprocess trees, because missed edges retain invalid descendants and spurious edges approach temporal rollback. *Object identity* must distinguish hierarchy and aliases beyond path strings; lexical comparison misses shared objects, while permanent inode merging invents dependencies after copy-up. *Selective reconstruction* must materialize a state that never existed temporally while preserving independent effects across repeated writes, deletions, frontier commits, and crashes.

These symptoms share one root cause. Existing substrates index speculative state by command, namespace, time, and pathname, but failures propagate through causal dependencies among versioned effects. Our key insight is to make that dependency structure the recovery unit. An *Agent Effect Transaction* (AET) executes one adaptive trajectory in a shared speculative view and rolls back only a failed effect's causal closure.

The design uses three core correctness mechanisms. An effect ledger captures READ, NEGATIVE, WRITE, and DELETE effects, derives cross-step dependencies, and normalizes hierarchy and supported aliases. Versioned per-step state supports selective reconstruction, while an overlap check rejects recoveries that would overwrite retained work. A policy-checked commit frontier and write-ahead log publish approved historical versions and recover interrupted materialization. Separately, a persistent `try` worker and incremental upperdir snapshots amortize execution; they are optimizations, not additional correctness challenges.

We implement these mechanisms in AGENTTX, a roughly 3.5 K-line Python/Linux prototype built on unprivileged `try`, OverlayFS, and `strace`. Across 144 controlled real-overlay

runs, causal rollback retained 100% of independent work and removed 100% of invalid descendants. At 48 document entries, it avoided 1,335.7 replay tokens versus an optimistic temporal checkpoint and 2,891.0 versus whole-session abort. The full 64-call path cost 148.5 ms/step, 43% below per-call isolation but about 3× unsafe bare execution. The checkpoint and abort results emulate recovery granularity; they are not external-system measurements.

This paper makes four contributions:

- We identify and quantify the recovery-unit mismatch in long coding-agent trajectories.
- We introduce AET, a recovery abstraction over causal dependencies among versioned effects.
- We build AGENTTX with opaque-command capture, selective reconstruction, durable publication, and amortized execution.
- We evaluate controlled DAGs, live-agent recovery, replay costs, crash injection, long sessions, and disjoint concurrency.

2 Background and Problem

2.1 Agent Trajectories and Effects

We model an agent trajectory as an ordered sequence of tool calls $T = \langle s_0, s_1, \ldots, s_n \rangle$. A tool call can be a structured API, a shell script, a compiler, or an arbitrary third-party program. The agent framework knows the boundary and requested arguments, but generally does not know the complete set of files read or changed by descendants of the process. The OS observes these effects but lacks the trajectory semantics needed to decide which state is approved or which later work depends on a mistake.

For each step, the runtime records effects E_i over workspace-visible names: successful reads (R), negative lookups (N), writes (W), and deletes (D). A later read can consume an earlier write. A negative lookup can influence a tool precisely because a path is absent. Directory writes and deletes can also change descendant reachability. These effects support dependency analysis, but pathname equality alone does not establish object identity.

2.2 Isolation and Recovery Granularity

Table 1 separates isolation from recovery semantics. Bare execution has neither. A per-call semisolate can discard one command, but it must explicitly carry successful state between calls and repeatedly pays setup cost. A session sandbox stages a whole trajectory but naturally commits or aborts the session. Temporal checkpoints retain a prefix. None of these selects a producer and its non-contiguous descendants while preserving independent later work.

Semisolates such as `try` can stage and inspect effects of an opaque command [2]. The unresolved problem lies across commands. Existing units organize provisional state by command, session, branch, or time, while a failure propagates through dependencies among effects. A trajectory runtime must recover that causal subgraph without discarding an unrelated suffix.

2.3 Goals and Scope

Goals. The transaction must keep the host unchanged before approval and preserve one continuous speculative view across tool calls. It must recover later work that is independent of a failure and durably publish approved effects. Deployment should require neither tool-specific rewrites nor a privileged daemon.

Scope. We target workspace-local filesystem effects of trusted but fallible coding agents. The model excludes irreversible network or cloud effects, process memory, and terminal state. It is not a confinement boundary for adversarial processes. These limits separate failure recovery from security mediation and whole-process checkpointing.

3 Observations and Motivation

3.1 Observation 1: Simple Isolation Is Safe but Costly

We first run the same deterministic 64-call coding workload under six modes. The workload includes repository exploration, a multi-file refactor, an injected failing CI path, an artifact derived from the faulty file, independent documentation and configuration changes, and a repair suffix. Bare execution is fastest at 49.7 ms per step but modifies the host immediately. Per-call `try` costs 260.7 ms per step. Full AGENTTX costs 148.5 ms per step and keeps the host clean, while the no-trace ablation costs 62.6 ms but cannot recover read-derived descendants correctly.

Figure 1 records the implementations before each hot-path optimization. Declared effects for trusted tools avoid unnecessary tracing; reusable command scripts, deferred blob collection, and direct script execution remove smaller repeated costs. A persistent worker reduces 393.6 to 151.5 ms per step by amortizing namespace and overlay setup. Incremental snapshot replay reduces the snapshot stage from 0.384 to 0.158 s, but we do not present that substage result as an equivalent endpoint speedup. These mechanisms make trajectory isolation practical; they are not additional correctness challenges.

3.2 Observation 2: Time Is Not Causality

Consider steps $s_0 : W(a)$, $s_1 : R(a), W(b)$, and $s_2 : W(c)$, followed by a failure attributed to s_0. A whole-session abort loses a, b, and c. A temporal rollback to before s_0 has the same outcome. The desired state removes a and derived b

Mode	Speculative unit	Native recovery unit	Later independent work	Host before approval
Bare	none	none	survives by immediate mutation	polluted
Per-call semisolate	one call	one call	no continuous shared state	clean
Session semisolate	trajectory	whole session	discarded on abort	clean
Temporal checkpoint	trajectory prefix	suffix after checkpoint	discarded with suffix	clean
AGENTTX	trajectory effect DAG	failed causal closure	retained when non-overlapping	clean

Table 1: **Isolation does not determine recovery granularity.** *Command, session, and temporal units cannot select a failed causal subgraph while retaining non-overlapping later work. Checkpoint and whole-session rows describe semantics, not external artifact results.*

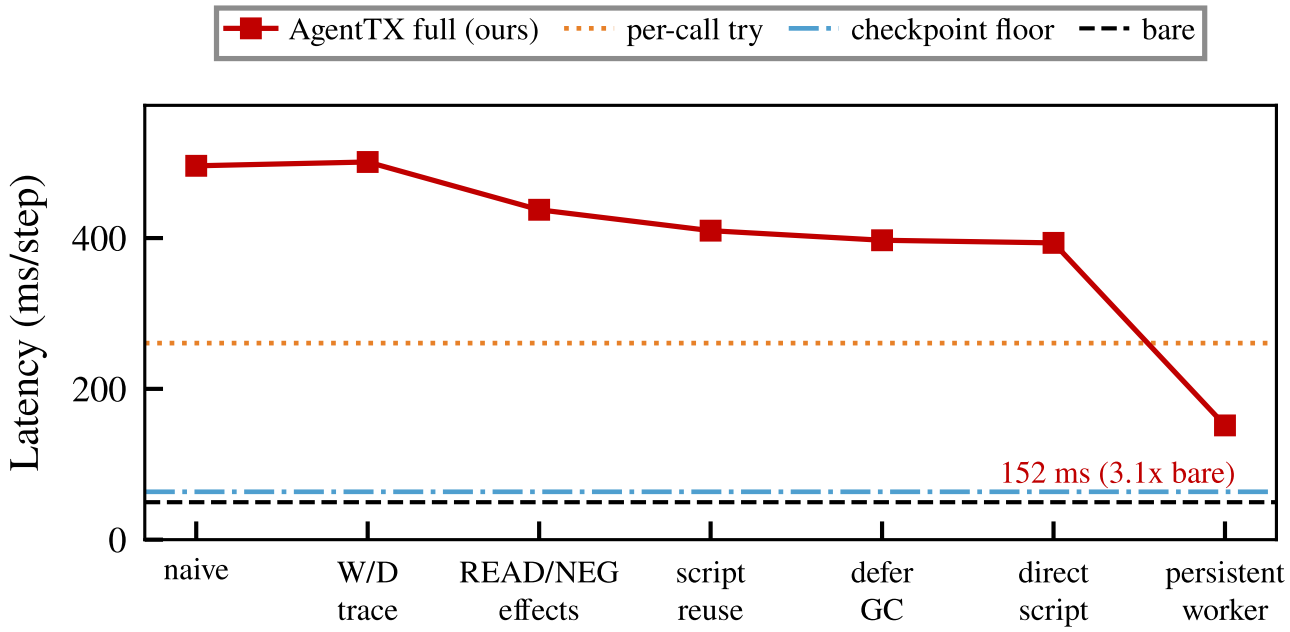

Figure 1: **Amortizing isolation dominates the optimization history.** *A persistent* `try` *worker removes repeated namespace setup and provides the largest endpoint improvement. Values are VM-local, and the final full path remains slower than unsafe bare execution.*

but retains independent c. Longer trajectories interleave many such branches, so the useful work lost by a time boundary grows with the distance between failure and diagnosis.

The mismatch persists at larger scale. In the controlled 64-call DAG, temporal recovery removes invalid work but retains only 41% of independent work; whole-session abort retains none. At the largest replay input, discarded valid work requires 1,335.7 tokens after optimistic temporal recovery and 2,891.0 after whole abort. File retention therefore determines user-visible replay work.

3.3 Three Challenges from One Root Cause

The root cause is a mismatch of indexes. Existing substrates index provisional state by command, namespace, time, and pathname, while an agent failure propagates through causal dependencies among versioned effects. Closing this mismatch creates three systems challenges.

Dependency discovery. Opaque commands hide their producer-consumer relationships. Missing an edge keeps an invalid descendant; adding every temporal edge degenerates to suffix rollback.

Object identity. Paths are names, not stable objects. Directory hierarchy, symlinks, hard links, renames, and OverlayFS copy-up can make lexical comparison either miss or invent dependencies.

Selective reconstruction. The desired post-recovery state may never have existed at a point in time. The runtime must undo a non-contiguous set, preserve later independent effects, handle repeated writes and deletions, and remain recoverable across a partial host commit.

The resulting insight is to use the trajectory as the speculation unit and the causal subgraph as the recovery unit. Dependency capture and reconstruction must be designed together: a selective policy over an incomplete graph is not

safe.

4 Agent Effect Transactions

4.1 Model and Goals

An Agent Effect Transaction is a tuple $\langle V, L, H, F \rangle$. V is the current speculative filesystem view. L records tool steps and their effects, while H provides historical states needed for reconstruction. F separates finalized steps from the speculative suffix. A tool call reads and mutates V, appends one node to L, and leaves the host unchanged.

The four components map directly to the goals in Section 2. V preserves continuous cross-call state while isolating the host. L identifies a causal recovery set, and H reconstructs it without discarding an independent suffix. F publishes only approved effects. AET defines these transitions without requiring a particular tracing, snapshot, or commit implementation.

4.2 Effect DAG

Each ledger node $s_i = (i, \text{tool}_i, E_i, P_i, \text{status}_i)$ contains effects and parent step identifiers. Let $\text{overlap}(p, q)$ hold when two effect names identify overlapping objects. For an active earlier step $j < i$, the transaction adds $j \in P_i$ when:

- s_j writes or deletes p, and s_i reads, negatively looks up, writes, or deletes an overlapping q; or
- s_j negatively looks up p, and s_i later writes overlapping q.

The first rule captures read-after-write and write serialization. The second captures assumptions based on absence. The identity relation is part of the AET contract; Section 5 states how AgentTX approximates it.

For a failed active step f, let $C_0 = \{f\}$. We compute

$$C_{k+1} = C_k \cup \{s_i \mid P_i \cap C_k \neq \emptyset\}, \qquad C(f) = \bigcup_{k \geq 0} C_k. \quad (1)$$

$C(f)$ is the least transitive descendant closure among active, uncommitted steps. Temporal rollback instead selects the suffix $\{s_i \mid i \geq f\}$.

4.3 Selective Reconstruction

Let $W(C)$ contain paths written or deleted by steps in $C(f)$. For each target, the runtime obtains a state that predates its earliest selected effect. It then checks every retained active effect against $W(C)$. If a retained effect overlaps a target, the runtime rejects recovery rather than guessing a merge. This fail-closed rule trades availability for a safety invariant:

> After a successful causal rollback, no effect written by $C(f)$ is visible, and every non-overlapping retained effect remains visible in the speculative view.

The transaction marks selected nodes rolled back only after reconstruction succeeds. Historical states for invalidated steps can then be reclaimed.

4.4 Commit Frontier

The frontier F is the largest finalized step identifier. Every step at or below F is approved for publication or durably marked rolled back. Advancing to $F' > F$ publishes active write/delete effects in $(F, F']$ and leaves later steps speculative. Rolled-back holes therefore do not block monotonic progress.

Publication and speculative recovery are separate transitions. An implementation must publish the frontier version even if a later speculative step rewrites the same object. AET requires durable recovery of that transition, but not atomic multi-path visibility to external observers.

5 AgentTX Design and Implementation

5.1 Tool-Boundary Effect Capture

Writes and deletes are obtained by fingerprinting the unmounted upper layer before and after a command. Fingerprints include content and metadata, thereby capturing repeated chmod, chown, touch, empty-directory, symlink, and whiteout effects. Linux OverlayFS may encode deletions as character-device or `.wh.*` whiteouts; both forms are normalized to D effects.

AgentTX captures successful reads and ENOENT/ENOTDIR lookups with `strace` [4]. The parser follows process creation, tracks working directories, and resolves modeled path operations relative to file descriptors. It records requested and resolved paths when a symlink changes the name. A failed trace invocation or parse aborts the step unless tracing is explicitly disabled. Coverage remains limited to modeled Linux pathname syscalls; it does not prove that every filesystem access is observed. Trusted tools may declare effects directly, making their declarations part of the trusted computing base.

5.2 Shared Semisolate and Persistent Worker

All calls in one session reuse a `try -N` upper layer. This shared layer provides the continuous speculative view required by AET. Later commands observe earlier uncommitted effects while the host remains unchanged. The launcher synchronizes `$PWD` with the protected workspace because a changed subprocess cwd alone can leave the environment inconsistent.

The persistent Python worker is a performance optimization over that design. It exchanges length-prefixed requests inside one semisolate to amortize namespace and overlay setup. If the worker dies, the current request uses the one-shot path; the following request starts a new worker. This fallback

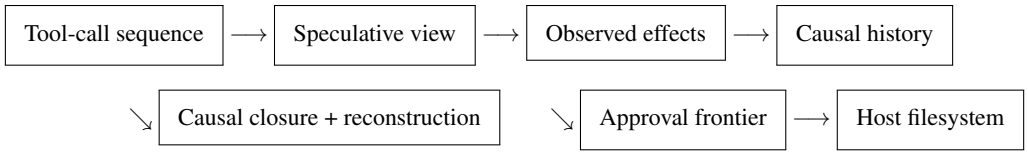

Figure 2: **Agent Effect Transaction lifecycle.** *A trajectory shares one speculative view, causal history selects recovery, and an approval frontier controls publication.*

preserves availability without making worker reuse part of AET correctness.

5.3 Snapshots and Aliases

Before step i, the layer store records `before_i`. Regular files are stored as immutable content-addressed blobs and hard-linked into snapshots. After the first step, an incremental snapshot clones the prior tree and replays only paths changed by the previous call. Native whiteouts and mode-000 trees are preserved. Causal rollback restores only selected logical paths; temporal rollback swaps the full upper tree to one pre-step image.

AgentTX treats ancestor-related paths as overlapping and records lexical and resolved paths for supported symlink accesses. Renames appear as path-level delete/create effects; the prototype lacks a stable object identifier across all namespace changes. Bind-mount equivalence is also outside the identity model.

Lower hard links expose a stronger substrate failure. On the tested Linux 5.15 OverlayFS path, writing one name copy-ups and splits the inode. A sibling can then read old content before AgentTX constructs the ledger. No additional edge can repair that lost visibility. Causal rollback therefore remains explicit until a more object-faithful substrate supplies stronger semantics.

5.4 Durable Commit and Policy

AgentTX uses anchored include filters to materialize selected frontier paths. If a later speculative step rewrites one path, AgentTX reconstructs the frontier version, publishes it, and restores the later upper-layer state. The runtime evaluates the persisted policy before WAL preparation or host mutation.

Before publication, the WAL saves affected host paths, the upper layer, the target frontier, and prior ledger state. Reload either restores the pre-commit state or finalizes a durable frontier. Session metadata uses same-directory atomic replace followed by file and directory fsync.

A shared allow/deny policy is enforced by the runtime, CLI, coding harness, and LLM agent. Policy rules persist across reload. The WAL provides crash recoverability, not external all-path atomic visibility: a concurrent observer can see a prefix while materialization is in progress.

6 Evaluation

Our evaluation asks:

RQ1 What overhead does trajectory isolation and dependency capture add (Section 6.2)?

RQ2 Does causal rollback remove invalid work while retaining independent work, and is dependency capture necessary (Section 6.3)?

RQ3 Can a live LLM use the recovery control plane, and does retained work reduce replay tokens (Sections 6.4 and 6.5)?

RQ4 Does the optimized runtime survive worker failure, reload, long sessions, and concurrent agents (Section 6.6)?

6.1 Setup and Methodology

Experiments run on the project VM with Linux 5.15, upstream `try`, OverlayFS, `strace`, and Python 3.11. Deterministic workloads report repeats, p50, and p95 where available. Real-agent experiments use `deepseek-chat`. All correctness experiments inspect the speculative view and physical host before and after the controlled commit.

We separate baselines by purpose. Bare execution is an unsafe latency lower bound because it mutates the host immediately. Per-call `try`, shared `try`, and the shared-snapshot path isolate execution components but lack equivalent causal recovery. AGENTTX without read tracing is a correctness ablation. Temporal checkpoint and whole-session abort are recovery-granularity emulations over the same real trajectory, not external artifact executions. Recent external systems remain qualitative because their FUSE, CRIU, kernel, compiler, or privilege requirements are unavailable on this VM.

6.2 RQ1: Runtime Overhead and Scaling

We expect shared execution to beat per-call isolation because it amortizes namespace setup. Full dependency capture should remain slower than unsafe bare execution because it traces reads and maintains reconstructable state. The current 64-call comparison is summarized in Table 2. Bare is not correctness-equivalent: it has a host pollution rate of one. Full AGENTTX is 43% faster than per-call isolation, but still 2.99× the bare per-step latency. The gap between full and no-trace reflects both system-call tracing and the larger read-effect graph.

At lengths 54, 64, and 96, full AGENTTX costs 156.4, 139.5, and 107.0 ms per step. Total wall time still grows

Mode	ms/step	Host clean
Bare	49.7	no
Per-call `try`	260.7	yes
Shared `try`	254.0	yes
Shared snapshot	63.5	yes
AGENTTX no-trace	62.6	yes
AGENTTX full	148.5	yes

Table 2: **Shared execution cuts per-call isolation cost.** *Full AgentTX is 43% faster than per-call `try` on the 64-call workload, but remains 2.99× slower than unsafe bare execution.*

from 8.45 to 10.27 s, so the decline reflects amortized fixed cost rather than faster longer runs. Step p50 is 19.6–24.4 ms, whereas p95 is 684.8–724.0 ms. Thus tracing and heavy tool boundaries still dominate the tail.

6.3 RQ2: Causal Retention and Dependency Ablation

We expect a correct causal policy to retain independent work and remove invalid descendants. A no-dependency ablation should retain apparent work but miss derived corruption. We execute real filesystem effect DAGs rather than simulate graph nodes. The sweep varies length (16/32/64), shape (chain/fan-out/layered), requested fault position (10/50/75%), and independent-work ratio (25/50/75%). Causal, temporal, and whole-session policies receive identical declared reads. The fourth mode deliberately removes read dependencies. Three repeats yield 144 real-overlay runs over 48 aggregated configurations; all keep the host clean before commit.

Figure 3 shows that full causal rollback retains 100% of independent work and removes 100% of invalid descendants in every configuration. At 64 calls, temporal rollback removes invalid work but retains only 41% of independent work; whole-session discard retains 0%. The no-dependency ablation retains independent files but removes only 4% of invalid work. Only dependency-aware causal rollback satisfies both objectives in every tested configuration. Causal rollback p95 at 64 calls is 272.7 ms.

6.4 RQ3a: Real-Agent Recovery

We ask whether a live tool-selecting model can identify a faulty producer and invoke the distinguishing recovery operation without a scripted target. We seed a repository with a faulty pipeline, an independent release note, a derived invalid artifact, and failing tests. A live LLM must inspect the ledger, select the earliest faulty producer, invoke causal rollback, retain the note, remove the artifact, repair the task, and pass tests. The model stops before publication. The benchmark verifies host cleanliness, applies policy, and performs the controlled commit.

Across three fresh sessions, root selection, target selection, independent retention, invalid removal, and tests succeed in all runs. The host leak rate is zero and wall p50/p95 is 29.0/30.8 s. This result establishes control-plane usability on one seeded task and one model, not on large repositories.

6.5 RQ3b: Avoided Replay Tokens

If causal recovery preserves valid artifacts, the model should not need to regenerate them. We therefore measure only replay caused by recovery granularity, not total session tokens. The token study isolates model work caused by recovery granularity. A fixed five-step trajectory contains two valid documents separated by a faulty producer and its descendants. Causal rollback retains both documents; an optimistic temporal checkpoint loses one; whole-branch abort loses two. A real LLM regenerates only missing documents at 12, 24, and 48 entries, with three repeats per cell.

All 27 samples succeed without a pre-commit host leak. Across the tested sizes, causal retention avoids 692–1,336 replay tokens relative to optimistic temporal recovery and 1,436–2,891 relative to whole-session abort. These are *avoided replay tokens* under recovery-granularity emulations. They are not total session-token savings or measurements of external systems.

6.6 RQ4: Robustness and Resource Cost

We expect the persistent fast path to survive worker loss without corrupting the session. We also test whether durable metadata scales beyond a short single-process trajectory.

An injected persistent-worker crash falls back to one-shot `try` for the current request and restarts the worker for the next. A 256-step session is closed at step 128, reloaded, completed, and commits all 256 files with step p50/p95 36.3/72.3 ms. Four concurrent agents, each writing 16 files in a separate workspace subtree, all commit without cross-contamination in 3.105 s. This test does not establish fencing for concurrent writes to the same path.

Content-addressed snapshots store 9.1 MB of unique blobs for 100.7 MB of logical snapshot payload (9.0%). This reduces repeated content but does not eliminate directory traversal, historical snapshot, or WAL-copy costs. Crash-injection tests also verify interrupted host materialization, persisted policies, native whiteouts, mode-000 trees, and metadata restoration.

A lower-hard-link probe exposes the current object-identity boundary. After OverlayFS copies up one name, its sibling reads stale data; selective commit then splits the inode pair. Ledger canonicalization cannot repair a read that already observed incorrect speculative state. Causal rollback therefore remains explicit on this substrate, and the concurrency result applies only to disjoint workspaces.

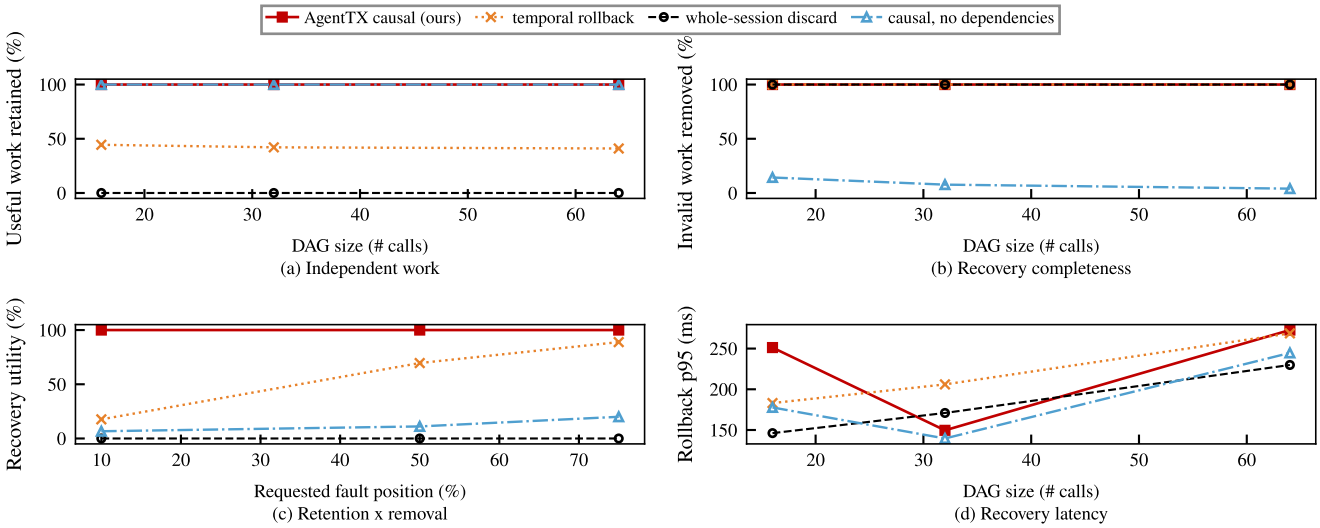

Figure 3: **Causal recovery requires both selection and dependency capture.** *Full causal recovery retains and cleans 100% in the controlled suite. At 64 calls, temporal recovery retains 41%, while the no-dependency ablation removes only 4% of invalid work.*

7 Discussion and Limitations

Why not make causal rollback the default? The explicit API is correct for path hierarchy and symlink aliases, but lower hard links split during OverlayFS copy-up on our substrate [3]. Because a sibling name can observe stale content before ledger analysis, default-on causal recovery requires a different FUSE/kernel substrate or stronger object mediation. Bind-mount coverage is also incomplete.

Why trace instead of infer commands? Command strings are not a reliable specification of transitive compiler, test, or script reads. System-call tracing observes actual accesses but adds workload-dependent cost and remains Linux-specific. An eBPF implementation could reduce collection overhead and improve deployment, but would not remove the need to define alias and unsupported-syscall semantics.

What state is outside the transaction? The current AET covers workspace-local filesystem effects. Network requests, emails, package registries, cloud control planes, process memory, and terminal state may be irreversible or require subsystem-specific compensation. Systems that capture full process/session state are complementary, not dominated by AGENTTX.

Is commit atomic? The WAL guarantees deterministic recovery of the session and host after an interrupted commit. It does not provide kernel-level all-path atomic visibility to external observers during materialization. Workloads requiring that property need a rename-based publication protocol or a transactional substrate.

How strong is the current evidence? Deterministic workloads cover up to 256 steps and controlled DAGs up to 64 calls. Real-agent runs are seeded and use one model. Several recent external systems cannot run on the current VM because of FUSE, CRIU, kernel, compiler, or privilege requirements. A final submission needs broader repositories, interleaved statistical runs, and artifact-level comparisons where available.

8 Related Work

AgentTX is closest to effect-capturing semisolates, agent-oriented filesystems, and checkpoint or branch runtimes. Its distinguishing recovery unit is a non-contiguous causal subgraph within one speculative trajectory. We compare that semantic choice below; without artifact-native measurements, we do not claim end-to-end superiority.

Effect capture and semisolates. `try` introduces unprivileged semisolates for opaque commands, including effect introspection, optional application, and stacking [2]. AGENTTX uses these mechanisms as its execution substrate. Its focus is the long-lived trajectory: automatic cross-step dependencies, non-contiguous causal recovery, frontier materialization, policy persistence, and commit WAL.

Agent-native filesystems and branches. YoloFS stages mutations, exposes snapshots, and uses progressive permission to improve agent/user control [8]. Its filesystem mediation and permission interface are broader than AGENTTX's current substrate. BranchFS provides nested copy-on-write branch contexts with atomic leaf commit/abort and first-commit-wins coordination [5]. These systems select snap-

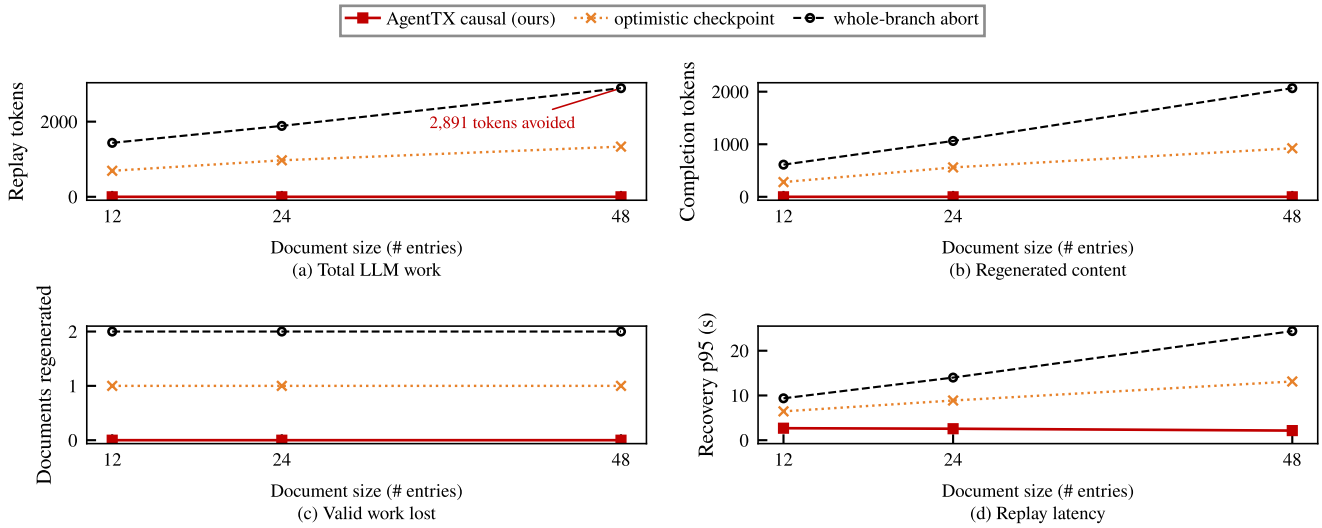

Figure 4: **Retained artifacts avoid LLM replay.** *Causal rollback requires no document-regeneration call. At 48 entries, optimistic temporal recovery replays 1,335.7 tokens and whole abort replays 2,891.0; zero excludes diagnosis, tests, and pre-failure tokens.*

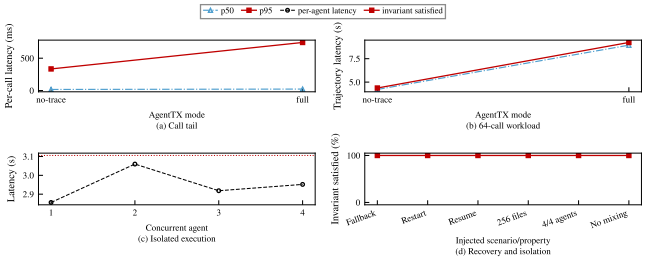

Figure 5: **The optimized path survives the tested failure modes.** *Worker fallback, 256-step reload, and four disjoint agents all complete correctly. Separate panels preserve the meaning of heterogeneous latency, correctness, and concurrency metrics.*

shots or branches; AGENTTX selects a causal subgraph inside one evolving trajectory.

Checkpoint and restore. Waypoint combines filesystem deltas with process, memory, shell, and terminal state for branchable sessions [7]. DeltaBox accelerates complete sandbox checkpoint/rollback using incremental filesystem and process state [1]. Crab observes turn-level effects to avoid checkpoints that do not contain recovery-relevant state [6]. Their state coverage or checkpoint efficiency is complementary to AGENTTX. AGENTTX instead answers which non-contiguous filesystem effects to remove after a failure has been identified.

9 Conclusion

Long-running coding agents need a transaction boundary larger than one command and a recovery unit smaller than one session. An Agent Effect Transaction changes that unit from a temporal boundary to a causal subgraph of versioned effects. AGENTTX realizes the abstraction with shared speculation, an effect ledger, selective reconstruction, and a durable commit frontier. In the current controlled suite, it retains all independent work, removes all invalid descendants, and avoids measurable LLM replay. The current claim remains filesystem-scoped and bounded by incomplete hard-link identity and external baseline coverage.

References

[1] Yunpeng Dong, Jingkai He, Shiqi Liu, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Baochuan Yang, Yubin Xia, and Haibo Chen. Deltabox: Scaling stateful ai agents with millisecond-level sandbox checkpoint/rollback, 2026.

[2] Evangelos Lamprou, Tianyu (Ezri) Zhu, Di Jin, Grigoris Ntousakis, Georgios Liargkovas, Calvin Eng, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Controlling Opaque-Component effects with semisolates and try. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 453–471. USENIX Association, 2026.

[3] The Linux Kernel Developers. Overlay filesystem. Linux kernel documentation, 2026.

[4] The strace developers. strace: Linux system call tracer, 2026.

[5] Cong Wang and Yusheng Zheng. Fork, explore, commit: Os primitives for agentic exploration, 2026.

[6] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes, 2026.

[7] Jiakai Xu, Tianle Zhou, Danielle Gillai, Ruizhe Fu, Georgios Liargkovas, Patrick Shen, Kostis Kaffes, and Eugene Wu. To-

ward systems foundations for agentic exploration. In *Systems for Agentic AI Workshop at SOSP*, 2025.

[8] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let ai agents yolo your files: Shifting information and control to filesystems for agent safety and autonomy, 2026.